DAM 0485 · Programación · Unidad Didáctica 4

Boletín 4.3: Retos Orientados a Objetos y Programación Competitiva

Retos avanzados de modelado orientado a objetos, algoritmos sobre estructuras de objetos y diseño de arquitecturas en Java 25 LTS.

Bienvenido al boletín de retos avanzados de la **Unidad 4: Clases y objetos**.

> 💡 ***Entrenamiento adicional recomendado:*** *Busca en CodeWars katas de nivel **7-6 kyu** etiquetadas con `Object-oriented Programming`, y en Acepta el Reto problemas de modelado y estructuración de datos. A continuación tienes **8 retos completos** diseñados con el mismo espíritu competitivo y de ingeniería de software, incluyendo pistas algorítmicas y soluciones íntegras comentadas en Java 25 LTS.*

Reto 1: La Baraja Española (40 cartas) y el Algoritmo de Fisher-Yates

Contexto y Planteamiento

Diseña un simulador completo de la baraja española de **40 cartas** (palos `"Oros"`, `"Copas"`, `"Espadas"` y `"Bastos"`; valores del `1` al `12` excluyendo el `8` y el `9`, o numerados del `1` al `10` para Sota, Caballo y Rey) aplicando tanto el enfoque clásico de clases (`Carta` y `Mazo`) como el enfoque moderno con `enum` y `record`:

1. Un `enum Palo` que defina los cuatro palos tradicionales (`OROS, COPAS, ESPADAS, BASTOS`).
2. Un `enum Valor` con los 10 valores de la baraja española y su puntuación numérica asociada.
3. Un `record Carta(Palo palo, Valor valor)` inmutable con un método `toString()` descriptivo (ej. `"As de OROS"`).
4. Una clase `Baraja` (o `Mazo`) que contenga el array de **40 objetos** `Carta`:
   - Debe inicializar las 40 combinaciones en el constructor (4 palos × 10 valores).
   - Método `public void barajar()`: reordena el array al azar mediante el **algoritmo in-place de Fisher-Yates / Knuth**, garantizando que todas las permutaciones sean equiprobables en tiempo lineal `O(n)` sin crear arrays auxiliares.
   - Método `public Carta robar()`: retira y devuelve la carta superior de la baraja (lanzando `IllegalStateException` si la baraja está agotada).
   - Método `public Carta[] repartir(int n)` (o `repartirMano(int n)`): devuelve un array con las primeras `n` cartas disponibles del mazo.

> **Pista para barajar (Algoritmo de Fisher-Yates)**
>
> Recorre el array de atrás hacia adelante (desde `i = cartas.length - 1` hasta `1`), y en cada posición intercambia la carta actual `cartas[i]` con una carta elegida en una posición aleatoria anterior o igual (`j` entre `0` e `i` inclusive).

▶ 💡 Ver Solución Comentada (Versión Moderna con Enum/Record y Versión Clásica)

Reto 2: Máquina Expendedora Automática con Algoritmo Voraz de Cambio

Contexto y Planteamiento

Diseña un sistema orientado a objetos para controlar una máquina expendedora automática con monedas reales:

1. `record Articulo(String codigo, String nombre, int precioCentimos, int stock)` (inmutable o con stock modificable).
2. Clase `MaquinaExpendedora`:
    - Gestiona un catálogo de artículos indexados por código.
    - Gestiona un saldo acumulado introducido por el cliente (en céntimos).
    - Método `public void insertarMoneda(int centimos)` (solo admite monedas de 5, 10, 20, 50 céntimos, 1€ y 2€).
    - Método `public Articulo comprar(String codigoArticulo)`: si el saldo es suficiente y hay stock, despacha el artículo, descuenta el stock y calcula el cambio restante.
    - Método `public int[] devolverCambio()`: descompone el cambio restante en el número mínimo de monedas de cada valor utilizando un **algoritmo voraz (*Greedy Algorithm*)** en denominaciones de 200, 100, 50, 20, 10 y 5 céntimos.

▶ 💡 Ver Solución Comentada

Reto 3: Sistema Bancario con Clientes y Múltiples Cuentas (Asociación Bidireccional)

Contexto y Planteamiento

Modela la relación entre un `Cliente` y sus `CuentasBancarias`:

- Un `Cliente` puede poseer hasta 3 cuentas bancarias distintas (ahorro, corriente, nómina).
- Método `vincularCuenta(CuentaBancaria c)` que asocie la cuenta al cliente evitando sobrepasar el límite de 3.
- Método `public double calcularPatrimonioTotal()` que compute la suma acumulada de saldos de todas las cuentas del cliente.
- Método `public static boolean transferir(CuentaBancaria origen, CuentaBancaria destino, double cantidad)`: operación atómica que verifique saldo, realice la retirada de origen y el abono en destino.

▶ 💡 Ver Solución Comentada

Reto 4: Calculadora de Polinomios Inmutables

Contexto y Planteamiento

Modela una clase inmutable `Polinomio` para manipular expresiones matemáticas del tipo $P(x) = c_0 + c_1x + c_2x^2 + \dots + c_nx^n$:

- El array interno de coeficientes almacena en el índice `i` el coeficiente del término de grado `i`.
- Constructor con array de coeficientes (realizando copia defensiva y recortando ceros superiores redundantes).
- Método `public double evaluar(double x)`: evalúa el valor numérico de `P(x)` mediante el **algoritmo de Horner** (`O(N)`).
- Método `public Polinomio sumar(Polinomio otro)`: devuelve un nuevo polinomio suma.
- Método `public Polinomio derivar()`: devuelve el polinomio derivada formal de `P(x)`.
- Sobrescribe `toString()` formateando algebraicamente la expresión matemática (ej. `"3.0x^2 - 4.0x + 5.0"`).

▶ 💡 Ver Solución Comentada

Reto 5: Simulación de Ecosistema en Tablero Matricial

Contexto y Planteamiento

Modela una simulación matricial de vida celular en un tablero de `FILAS × COLUMNAS`:

- `enum Especie { VACIA, HIERBA, CONEJO }`
- Cada casilla contiene un objeto `Organismo(Especie especie, int energia)`.
- En cada turno de la simulación:
  1. La `HIERBA` gana 1 punto de energía.
  2. El `CONEJO` pierde 1 punto de energía. Si su energía llega a 0, muere (se convierte en `VACIA`).
  3. Si un `CONEJO` tiene adyacente una casilla con `HIERBA`, se desplaza a ella, devora la hierba y suma su energía.
- Diseña la clase `TableroEcosistema` y simula 5 ciclos mostrando la evolución en consola.

▶ 💡 Ver Solución Comentada

Reto 6: Cola de Impresión Segura con Prioridades (Singleton + FIFO)

Contexto y Planteamiento

En sistemas operativos y servidores de red, el gestor de impresión (*Print Spooler*) debe ser único en toda la máquina (Patrón Singleton) y procesar trabajos ordenados según una cola de prioridad:

1. `record Documento(String id, String nombre, int paginas, int prioridad)` (prioridad 1 = Urgente, 2 = Normal, 3 = Baja).
2. Clase `SpoolerImpresion` (Singleton):
   - Constructor estrictamente privado.
   - Array interno de trabajos pendientes con capacidad máxima para 20 documentos.
   - Método `public boolean encolar(Documento doc)`: inserta el documento de forma ordenada según su prioridad (menor número = mayor prioridad) respetando el orden de llegada para igual prioridad.
   - Método `public Documento imprimirSiguiente()`: extrae y procesa el trabajo con máxima prioridad.
   - Método `public int getTrabajosPendientes()`.

▶ 💡 Ver Solución Comentada

Reto 7: Álgebra de Conjuntos con Varargs (`ConjuntoEnteros`) y Análisis de Complejidad

Contexto y Planteamiento

Implementa una estructura de datos propia llamada `ConjuntoEnteros` que represente un conjunto matemático finito de números enteros **sin elementos repetidos**, respaldado internamente por un array dinámico `int[]`:

1. **Constructor con `varargs`** (`public ConjuntoEnteros(int... elementos)`): crea el conjunto a partir de una secuencia arbitraria de enteros (o vacío si no se pasan argumentos), descartando automáticamente cualquier valor duplicado.
2. **Operaciones básicas y con `varargs`**:
   - `public int getTamano()` y `public boolean vacio()`.

- `public boolean pertenece(int... elementos)`: devuelve `true` si **todos** los elementos pasados por parámetro pertenecen al conjunto.
- `public boolean inserta(int... elementos)`: añade los elementos que aún no pertenezcan al conjunto, redimensionando el array interno solo en la cantidad exacta necesaria. Devuelve `true` si el conjunto ha cambiado (al menos un elemento nuevo insertado).
- `public boolean borra(int... elementos)`: elimina del conjunto todos los elementos indicados que estén presentes y compacta el array. Devuelve `true` si el conjunto se ha modificado.

3. **Operaciones algebraicas inmutables** (devuelven un nuevo `ConjuntoEnteros` sin alterar los operandos):
   - `public ConjuntoEnteros union(ConjuntoEnteros otro)`: elementos presentes en `this` o en `otro` (`A ∪ B`).
   - `public ConjuntoEnteros interseccion(ConjuntoEnteros otro)`: elementos comunes a ambos conjuntos (`A ∩ B`).
   - `public ConjuntoEnteros menos(ConjuntoEnteros otro)`: diferencia de conjuntos (`A \ B`), es decir, elementos de `this` que no pertenecen a `otro`.
4. **Identidad de `Object`**: sobrescribe `toString()` (ej. `"{1, 4, 7}"`), `equals(Object obj)` (dos conjuntos son iguales si tienen el mismo tamaño y exactamente los mismos elementos, **sin importar el orden** en que estén almacenados) y `hashCode()` (cuyo cálculo debe ser conmutativo para que el orden no altere el hash).
5. **Pregunta de reflexión algorítmica:** ¿Cuál es el coste temporal en notación asintótica (`O(...)`) de `pertenece`, `union`, `interseccion` y `equals` sobre un array desordenado frente a mantener el array interno ordenado y usar búsqueda binaria?

▶ 💡 Ver Solución Comentada y Análisis de Complejidad

Reto 8: Clínica Dietista (Cálculo de IMC) y Validador de Cuenta Bancaria Española (CCC)

Contexto y Planteamiento

Diseña un sistema integral de gestión de pacientes para una clínica de nutrición que combine el modelado de dominio con el algoritmo oficial de validación bancaria española:

1. **Clase `Paciente` y `enum Sexo { HOMBRE, MUJER }`**:
   - Atributos privados: `numPaciente` (código alfanumérico de 8 cifras + letra generado automáticamente al instanciar mediante un método privado `generaNumPaciente()`), `nombre`, `edad`, `sexo` (por defecto `MUJER`), `peso` (kg) y `altura` (metros), más su número de cuenta domiciliada `ccc`.
   - Constantes estáticas de diagnóstico nutricional: `INFRAPESO = -1`, `PESO_IDEAL = 0` y `SOBREPESO = 1`.
   - Tres constructores encadenados con `this(...)`: por defecto, con `(nombre, edad, sexo)` y completo con todos los parámetros.
   - Método `public int calcularIMC()`: calcula `IMC = peso / (altura²)` y devuelve `INFRAPESO` (`< 20`), `PESO_IDEAL` (`20` a `25` incluidos) o `SOBREPESO` (`> 25`).
   - Método `public boolean esMayorDeEdad()`: devuelve `true` si `edad >= 18`.
2. **Validador de Código Cuenta Cliente (`EEEE-SSSS-DC-CCCCCCCCCC`)**:
   - Método estático `public static boolean validarCCC(String ccc)` que verifique mediante expresión regular el formato de 20 dígitos (`4` de entidad, `4` de sucursal, `2` dígitos de control y `10` de número de cuenta) y compruebe matemáticamente los **dos dígitos de control (DC)** usando el algoritmo oficial de pesos módulo 11 (`{1, 2, 4, 8, 5, 10, 9, 7, 3, 6}`).

▶ 💡 Ver Solución Comentada